

NextJS

4.10 - Rotas Dinâmicas: A Página de Detalhes

Objetivo de Aprendizagem

Dominar a criação de rotas dinâmicas utilizando colchetes `[id]`, aprendendo a capturar parâmetros da URL e utilizá-los para buscar dados específicos de um único produto na API.

Parte 1: Conceitos e Fundamentos

O que são Rotas Dinâmicas?

Imagine que você tem milhares de pratos no banco de dados. Seria impossível criar uma pasta para cada um. As **Dynamic Routes** permitem que você defina um "coringa" na estrutura de pastas usando colchetes: `[slug]`, `[id]` ou `[nome]`.

- **Estrutura:** `app/cardapio/[id]/page.js`
- **URL gerada:** `/cardapio/58997c7c...` ou `/cardapio/30a967b6...`

Como o Next.js entrega o ID?

Todo componente dentro de uma pasta de rota dinâmica recebe automaticamente uma propriedade (prop) chamada `params`. Ela contém um objeto com o valor exato que o usuário digitou ou clicou na URL.

Parte 2: Prática - Criando a Ficha Técnica do Prato

Vamos construir a página que exibe os detalhes completos de um produto da sua API.

Passo 1: Criando a Pasta e o Arquivo

1. Dentro da pasta `src/app/cardapio`, crie uma nova pasta chamada `[id]` (com os colchetes).
2. Dentro de `src/app/cardapio/[id]`, crie o arquivo `page.js`.

Passo 2: Implementando o Fetch por ID

```
// Nesta página, usaremos o endpoint individual da sua API: // //  
//https://api-restaurante-5iqb.onrender.com/api/produtos/{id}.
```

```
import Image from 'next/image';
```

```
import Link from 'next/link';
```

```
import BotaoFavorito from "@components/BotaoFavorito";
```

```
export default async function DetalhePrato({ params }) {
```

```
  // 1. Capturamos o ID da URL (aguardamos o params pois ele é assíncrono no Next 15+)
```

```
  const { id } = await params;
```

```
  // 2. Buscamos os dados desse prato específico
```

```
  const res = await fetch(`https://api-restaurante-5iqb.onrender.com/api/produtos/${id}`, {
```

```
    cache: 'no-store' // SSR: Sempre buscar detalhes atualizados
```

```
  });
```

```
  const prato = await res.json();
```

```
  return (
```

```
    <main className="max-w-5xl mx-auto p-10">
```

```
      {/* Botão de Voltar */}
```

```
      <Link href="/cardapio" className="text-orange-600 hover:underline mb-6 inline-block">
```

```
        ← Voltar para o Cardápio
```

```
      </Link>
```

```
      <div className="grid grid-cols-1 md:grid-cols-2 gap-12 mt-4">
```

```
        {/* Lado Esquerdo: Imagem Otimizada */}
```

```
        <div className="relative h-[400px] rounded-3xl overflow-hidden shadow-2xl">
```

```
          <Image
```

```
            src={prato.imagem}
```

```
            alt={prato.nome}
```

```
            fill
```

```
            className="object-cover"
```

```
            priority
```

```
          />
```

```
        </div>
```

```
        {/* Lado Direito: Informações da API */}
```

```
        <div className="flex flex-col justify-center">
```

```
          <span className="text-orange-600 font-semibold uppercase tracking-widest text-sm">
```

```
            {prato.categoria}
```

```
          </span>
```

```
          <h1 className="text-5xl font-bold text-gray-800 mt-2">{prato.nome}</h1>
```

```
          <p className="text-gray-600 mt-6 text-lg leading-relaxed">
```

```
            {prato.descricao}
```

```
          </p>
```

```

<div className="mt-8 p-6 bg-gray-50 rounded-2xl flex items-center justify-between">
  <div>
    <p className="text-gray-400 text-xs uppercase font-bold">Preço Individual</p>
    <span className="text-4xl font-bold text-green-700">
      R$ {prato.preco.toFixed(2)}
    </span>
  </div>
  <BotaoFavorito />
</div>

<p className="text-[10px] text-gray-300 mt-4 font-mono uppercase">
  ID do Sistema: {prato.id}
</p>
</div>
</div>
</main>
);
}

```

Passo 3: Conectando a Lista ao Detalhe

Para que o usuário consiga chegar nesta página, precisamos atualizar o **Link** na nossa lista geral de produtos.

1. Abra `src/app/cardapio/page.js`.
2. Envolve o título ou o card do prato com um **Link** que aponte para o ID correto:

```

// Dentro do map em app/cardapio/page.js
<Link href={`/${cardapio}/${prato.id}`} className="hover:opacity-95 transition-opacity">
  <h2 className="text-xl font-bold text-gray-800">{prato.nome}</h2>
</Link>

```

Ficando assim:

```

import Image from 'next/image';
import Link from 'next/link';
import BotaoFavorito from "@/components/BotaoFavorito";

export default async function PaginaCardapio() {

  const resposta = await
  fetch('https://api-restaurante-5iqb.onrender.com/api/produtos', {
    next: { revalidate: 60 }
  })
  .then(res => res.json())
  .catch(err => console.log(err));

  return (
    <div>
      <h1>Cardápio</h1>
      <div>
        {resposta.map((prato) => (
          <div key={prato.id}>
            <Image alt={prato.imagem} />
            <div>
              <h2>{prato.nome}</h2>
              <p>{prato.preco}</p>
            </div>
            <BotaoFavorito prato={prato} />
          </div>
        ))}
      </div>
    </div>
  );
}

```

```

});
// Convertendo para JSON
const produtos = await resposta.json();

return (
  <main className="p-10 max-w-7xl mx-auto">
    <div className="flex justify-between items-end mb-10">
      <div>
        <h1 className="text-4xl font-bold text-gray-800 font-serif">Nosso Cardápio</h1>
        <p className="text-gray-500 mt-2">Pratos artesanais preparados com ingredientes frescos.</p>
      </div>
      <span className="text-sm bg-gray-100 px-3 py-1 rounded-full text-gray-400">
        {produtos.length} itens encontrados
      </span>
    </div>

    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
      {produtos.map((prato) => {
        // Trata URLs vindas da API com caracteres inválidos '<' e '>' ou espaçamentos
        const urlImagemLimpa = prato.imagem?.replace(/[<>]/g, '').trim();

        return (
          <div key={prato.id} className="bg-white rounded-2xl shadow-md overflow-hidden flex flex-col border border-gray-100 hover:shadow-xl transition-shadow">

            <div className="relative h-56 w-full">
              {urlImagemLimpa ? (
                <Image
                  src={urlImagemLimpa}
                  alt={prato.nome || "Foto do prato"}
                  fill
                  className="object-cover"

```

```

        sizes="(max-width: 768px) 100vw, (max-width: 1200px)
50vw, 33vw"

        unoptimized // Evita erros se a API retornar
domínios não mapeados no next.config.js
    />
    ) : (
        <div className="w-full h-full bg-gray-200 flex
items-center justify-center
text-gray-400 text-sm">
            Sem Imagem
        </div>
    ) }

    {prato.destacado && (
        <span className="absolute top-4 left-4 bg-orange-600
text-white text-xs
font-bold px-3 py-1 rounded-full shadow-lg">
            ESTRELA DA CASA ★
        </span>
    ) }
</div>

<div className="p-6 flex flex-col flex-grow">
    <div className="flex justify-between items-start mb-2">
        <Link href={`/${cardapio/${prato.id}}`}
className="hover:opacity-95 transition-opacity">
            <h2 className="text-xl font-bold
text-gray-800">{prato.nome}</h2>
        </Link>

        <span className="text-xs font-semibold text-orange-600
uppercase
tracking-wider">
            {prato.categoria}
        </span>
    </div>

    <p className="text-gray-500 text-sm mb-6 line-clamp-2">
        {prato.descricao}
    </p>

```

```

        <div className="mt-auto flex items-center
justify-between border-t pt-4">
            <span className="text-2xl font-bold text-green-700">
                R$ {Number(prato.preco || 0).toFixed(2)}
            </span>
            <BotaoFavorito />
        </div>
    </div>
</div>
);
})}
</div>
</main>
);
}

```

Passo 4: O Teste de Navegação

Faça esse teste:

1. Abrir a listagem de cardápio (</cardapio>).
2. Clicar no nome de um prato (ex: "Bruschetta Caprese").
3. Observar a URL: Ela deve mudar para algo como </cardapio/1da03340>...
4. Ver a página de detalhes carregando os dados corretos daquele prato específico.

Resumo da Prática

-  **Pasta [id]:** Aprendemos que colchetes criam rotas flexíveis.
-  **params:** Capturamos o "caminho" da URL para saber qual prato exibir.
-  **Fetch Dinâmico:** Concatenamos o ID na URL da API para buscar o dado exato.
-  **UX:** Adicionamos um botão de "Voltar" para facilitar a navegação do cliente.

4.11 - Loading States e Skeleton Screens

Objetivo de Aprendizagem

Aprender a utilizar o arquivo especial `loading.js` para criar transições suaves e estados de carregamento (Skeletons) que melhoram a percepção de velocidade do usuário.

Parte 1: Conceitual e Fundamentos

O que é o Streaming?

Imagine que o servidor é a cozinha. Sem o **Streaming**, o garçom (navegador) só traz o prato quando tudo estiver pronto. Com o Streaming, ele traz logo as bebidas e os talheres (Layout/Navbar) e, assim que o prato principal (Dados da API) estiver pronto, ele coloca-o na mesa.

O arquivo `loading.js`

No App Router, se colocar um arquivo chamado `loading.js` dentro de uma pasta, o Next.js irá exibi-lo automaticamente enquanto o `page.js` daquela mesma pasta estiver processando o `await fetch`.

- **Vantagem:** Não precisa criar estados de `loading` manuais dentro do componente.
 - **Skeleton Screens:** Em vez de um ícone de "carregando" (spinner), usamos blocos cinzentos que imitam o formato do conteúdo real, reduzindo a ansiedade do usuário.
-

Parte 2: Prática - Skeleton para o Cardápio

Vamos criar um carregamento elegante para a nossa lista de pratos.

Passo 1: Criando o arquivo de Loading

1. Dentro da pasta `src/app/cardapio/`, crie o arquivo `loading.js`.

Passo 2: Desenhando o Skeleton com Tailwind

Vamos usar a classe `animate-pulse` do Tailwind para criar o efeito de "pulsação" nos blocos.

```
export default function LoadingCardapio() {  
  // Criamos um array de 6 itens para simular os cards carregando  
  const skeletons = Array.from({ length: 6 });  
  
  return (  

```

```

<main className="p-10 max-w-7xl mx-auto">
  <div className="h-10 w-48 bg-gray-200 animate-pulse rounded-md mb-10"></div>

  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
    {skeletons.map((_, index) => (
      <div key={index} className="border border-gray-100 rounded-2xl overflow-hidden
shadow-sm bg-white">
        {/* Bloco da Imagem */}
        <div className="h-56 bg-gray-200 animate-pulse w-full"></div>

        <div className="p-6">
          {/* Bloco do Título e Categoria */}
          <div className="flex justify-between mb-4">
            <div className="h-6 w-32 bg-gray-200 animate-pulse rounded"></div>
            <div className="h-4 w-16 bg-gray-100 animate-pulse rounded-full"></div>
          </div>

          {/* Bloco da Descrição */}
          <div className="space-y-2 mb-6">
            <div className="h-3 w-full bg-gray-100 animate-pulse rounded"></div>
            <div className="h-3 w-3/4 bg-gray-100 animate-pulse rounded"></div>
          </div>

          {/* Bloco do Preço e Botão */}
          <div className="flex justify-between items-center pt-4 border-t border-gray-50">
            <div className="h-8 w-24 bg-gray-200 animate-pulse rounded"></div>
            <div className="h-10 w-28 bg-gray-200 animate-pulse rounded-lg"></div>
          </div>
        </div>
      </div>
    ))}
  </div>
</main>
);
}

```

Passo 3: Testando o Comportamento

Para ver o efeito funcionar (já que a API é rápida), pode simular um atraso no arquivo `src/app/cardapio/page.js`:

1. Abra o `page.js` do cardápio.
2. Adicione um "atraso artificial" antes do fetch:

```
// Apenas para teste visual
```



```
await new Promise((resolve) => setTimeout(resolve, 3000));
const resposta = await fetch(...);
```

O que vai acontecer?

Ao clicar no link "Cardápio" na Navbar:

1. A Navbar **não desaparece** (o Layout é preservado).
2. O conteúdo central muda instantaneamente para os blocos cinzentos pulsantes.
3. Após 3 segundos, os blocos são substituídos pelos pratos reais da API.

Diferença Visual: Spinner vs Skeleton

Tipo de Loading	Experiência do Usuário (UX)
Página em Branco	❌ Péssima (Usuário acha que o site quebrou).
Spinner (Círculo)	⚠️ Regular (O usuário sabe que está carregando, mas não sabe o quê).
Skeleton Screen	✅ Excelente (Prepara o olhar do usuário para onde a informação vai estar).

Resumo

- ✅ O arquivo **loading.js** é ativado automaticamente durante qualquer **await** no servidor.
- ✅ O **Streaming** mantém o Layout fixo e carrega apenas o conteúdo dinâmico.
- ✅ **Skeleton Screens** (com **animate-pulse**) transmitem uma sensação de profissionalismo e velocidade.
- ✅ No Next.js, a interface de carregamento é tratada como uma parte importante do design, não como um erro.

4.12 - Error Handling: Lidando com Falhas na API

Objetivo de Aprendizagem

Aprender a utilizar o arquivo especial `error.js` para criar **Error Boundaries** automáticos, garantindo que uma falha na busca dos pratos não derrube a navegação do site todo.

Parte 1: Conceitual e Fundamentos

O que é um Error Boundary?

Imagine que o lustre de uma mesa do restaurante queima. Você não fecha o restaurante inteiro, certo? Você apenas isola aquela mesa enquanto conserta o problema.

No Next.js, o arquivo `error.js` faz exatamente isso:

- **Isolamento:** Se a página de cardápio der erro, a `Navbar` e o restante do layout continuam funcionando.
- **Recuperação:** O componente de erro recebe uma função chamada `reset()`, que tenta renderizar a página novamente sem que o usuário precise dar F5 no navegador.

Importante: Diferente de outros arquivos especiais, o `error.js` precisa **obrigatoriamente** ser um **Client Component**, pois ele precisa interagir com o navegador para tentar recuperar a página.

Parte 2: Prática - Lidando com o Restaurante Offline

Vamos criar uma tela de erro amigável para quando a nossa API de pratos falhar.

Passo 1: Criando o arquivo de Erro

1. Dentro da pasta `src/app/cardapio/`, crie o arquivo `error.js`.

Passo 2: Implementando a interface de recuperação

Vamos criar um componente que informe o problema e ofereça um botão de "Tentar Novamente".

"use client"; //  Obrigatório para arquivos de erro

```
import { useEffect } from 'react';
```

```
export default function ErrorCardapio({ error, reset }) {
```

```
  useEffect(() => {
```

```

// Aqui você poderia enviar o erro para um serviço de log (ex: Sentry)
console.error("Erro capturado no cardápio:", error);
}, [error]);

return (
  <main className="flex flex-col items-center justify-center min-h-[60vh] p-10 text-center">
    <div className="bg-red-50 p-8 rounded-3xl border border-red-100 max-w-md">
      <h2 className="text-4xl mb-4">👨‍🍳⚠️ </h2>
      <h1 className="text-2xl font-bold text-gray-800">Ops! A cozinha está em
manutenção.</h1>
      <p className="text-gray-600 mt-4">
        Não conseguimos carregar os pratos neste momento. Nossa equipe já foi avisada!
      </p>

      <div className="mt-8 flex flex-col gap-3">
        <button
          onClick={() => reset()} // Tenta renderizar a página novamente
          className="bg-orange-600 text-white px-6 py-3 rounded-xl font-bold
hover:bg-orange-700 transition-colors"
        >
          Tentar Novamente
        </button>

        <button
          onClick={() => window.location.reload()}
          className="text-gray-400 text-sm hover:underline"
        >
          Recarregar página inteira
        </button>
      </div>
    </div>
  </main>
);
}

```

Passo 3: Como testar o Erro?

Para ver essa tela em ação sem precisar derrubar a API real, você pode forçar um erro no seu código:

1. Abra `src/app/cardapio/page.js`.
2. Adicione uma linha que quebre a execução:

```

export default async function PaginaCardapio() {
  // Forçando um erro para teste

```

```
// throw new Error("API Offline");

const resposta = await fetch(...);
// ... rest of the code
}
```

Ficando assim o código:

```
import Image from 'next/image';
import Link from 'next/link';
import BotaoFavorito from "@/components/BotaoFavorito";

export default async function PaginaCardapio() {
  //Forçando um erro para teste (descomente a linha abaixo para
  simular)
  throw new Error("API Offline");

  await new Promise((resolve) => setTimeout(resolve, 3000));

  const resposta = await
fetch('https://api-restaurante-5iqb.onrender.com/api/produtos', {
  next: { revalidate: 60 }
});
  // Convertendo para JSON
  const produtos = await resposta.json();

  return (
    <main className="p-10 max-w-7xl mx-auto">
      <div className="flex justify-between items-end mb-10">
        <div>
          <h1 className="text-4xl font-bold text-gray-800
font-serif">Nosso Cardápio</h1>
          <p className="text-gray-500 mt-2">Pratos artesanais preparados
com ingredientes
frescos.</p>
        </div>
        <span className="text-sm bg-gray-100 px-3 py-1 rounded-full
text-gray-400">
          {produtos.length} itens encontrados
        </span>
      </div>
```

```

    <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3
gap-8">
      {produtos.map((prato) => {
        // Trata URLs vindas da API com caracteres inválidos '<' e '>'
        ou espaçamentos
        const urlImagemLimpa = prato.imagem?.replace(/[<>]/g,
        '').trim();

        return (
          <div key={prato.id} className="bg-white rounded-2xl
shadow-md overflow-hidden
flex flex-col border border-gray-100 hover:shadow-xl
transition-shadow">

            <div className="relative h-56 w-full">
              {urlImagemLimpa ? (
                <Image
                  src={urlImagemLimpa}
                  alt={prato.nome || "Foto do prato"}
                  fill
                  className="object-cover"
                  sizes="(max-width: 768px) 100vw, (max-width: 1200px)
50vw, 33vw"
                  unoptimized // Evita erros se a API retornar
domínios não mapeados no next.config.js
                />
              ) : (
                <div className="w-full h-full bg-gray-200 flex
items-center justify-center
text-gray-400 text-sm">
                  Sem Imagem
                </div>
              )}

              {prato.destacado && (
                <span className="absolute top-4 left-4 bg-orange-600
text-white text-xs
font-bold px-3 py-1 rounded-full shadow-lg">
                  ESTRELA DA CASA ★
                </span>
              )}
            </div>
          </div>
        )
      )}
    </div>

```

```

    ) }
  </div>

  <div className="p-6 flex flex-col flex-grow">
    <div className="flex justify-between items-start mb-2">
      <Link href={`/${cardapio/${prato.id}}`}
className="hover:opacity-95 transition-opacity">
        <h2 className="text-xl font-bold
text-gray-800">{prato.nome}</h2>
      </Link>

      <span className="text-xs font-semibold text-orange-600
uppercase
tracking-wider">
        {prato.categoria}
      </span>
    </div>

    <p className="text-gray-500 text-sm mb-6 line-clamp-2">
      {prato.descricao}
    </p>

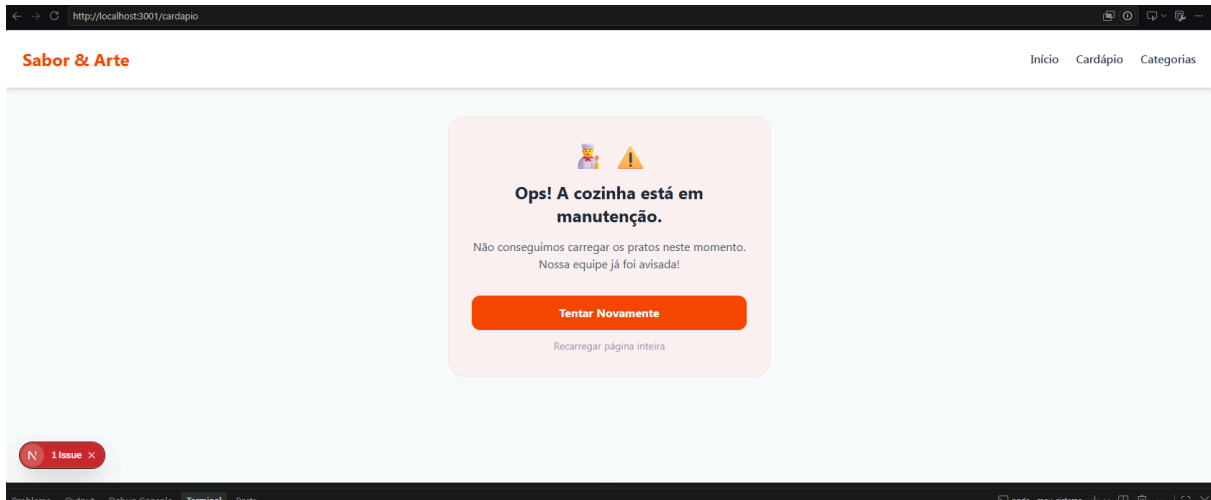
    <div className="mt-auto flex items-center
justify-between border-t pt-4">
      <span className="text-2xl font-bold text-green-700">
        R$ {Number(prato.preco || 0).toFixed(2)}
      </span>
      <BotaoFavorito />
    </div>
  </div>
</div>
);
}) }
</div>
</main>
);
}

```

O que você vai notar:

- Ao forçar o erro, a tela de "Cozinha em manutenção" aparece apenas na área do conteúdo.

- A **Navbar** continua lá em cima, permitindo que o usuário navegue para a "Home" ou "Categorias" mesmo com o cardápio quebrado.



Hierarquia de Erros

Se você tiver um `error.js` dentro de `app/cardapio/` e outro na raiz `app/`, o Next.js sempre usará o **mais próximo** da falha. Isso permite criar mensagens personalizadas: "Erro ao carregar pratos" vs "Erro genérico no sistema".

Arquivo

Alcance

`app/cardapio/error.js` Erros apenas dentro da rota de cardápio.

`app/layout.js` (global) Para erros fatais que quebram o site inteiro (use `global-error.js`).

Resumo

-  **Isolamento:** Erros em rotas filhas não quebram o Layout pai.
-  **Client-side:** O arquivo `error.js` deve ser `"use client"`.
-  **Resiliência:** A função `reset()` permite que o usuário tente carregar os dados novamente sem recarregar o site.
-  **UX Profissional:** Sair do "Erro 500" genérico para uma mensagem humanizada que faça sentido para o cliente do restaurante.

OBS: Para darmos sequência, pode remover a seguinte linha que força o erro

```
throw new Error("API Offline");
```

4.13 - Busca e Filtros (Query Params)

Objetivo de Aprendizagem

Aprender a manipular os parâmetros de busca da URL (`searchParams`) para filtrar os pratos da API do restaurante de forma dinâmica e performática.

Parte 1: Conceitual e Fundamentos

O que são Query Params?

São as informações que aparecem após a interrogação na URL:

`restaurante.com/cardapio?categoria=Bebidas`.

No Next.js (App Router), as páginas de servidor (`page.js`) recebem automaticamente uma prop chamada `searchParams`. Ao contrário do `params` (que é para a rota fixa), o `searchParams` é para filtros opcionais.

Por que usar a URL para filtrar?

- **Estado Compartilhável:** O usuário pode favoritar a página já filtrada.
 - **Histórico do Navegador:** O botão "Voltar" do navegador funciona para desfazer um filtro.
 - **SEO:** O Google consegue indexar páginas de categorias específicas separadamente.
-

Parte 2: Prática - Filtrando o Cardápio Real

Vamos criar um campo de busca e botões de categoria que atualizam a nossa lista de pratos.

Passo 1: Criando o Componente de Busca (Client Side)

Como precisamos capturar o que o usuário digita, este componente deve ser `"use client"`.

1. Crie o arquivo `src/components/Busca.js`:

```
"use client";

import { useRouter, usePathname, useSearchParams } from 'next/navigation';

export default function Busca() {
  const searchParams = useSearchParams();
  const pathname = usePathname();
  const { replace } = useRouter();

  function handleSearch(termo) {
    const params = new URLSearchParams(searchParams);
    if (termo) {
      params.set('busca', termo);
    } else {
      params.delete('busca');
    }
    // Atualiza a URL sem recarregar a página (ex: /cardapio?busca=pizza)
    replace(`${pathname}?${params.toString()}`);
  }

  return (
    <div className="mb-8">
      <input
        type="text"
        placeholder="🔍 Buscar um prato..."
        className="w-full p-4 rounded-2xl border border-gray-200 shadow-sm focus:ring-2 focus:ring-orange-500 outline-none"
        onChange={(e) => handleSearch(e.target.value)}
        defaultValue={searchParams.get('busca')?.toString()}
      />
    </div>
  );
}
```

Passo 2: Atualizando a Página de Cardápio (Server Side)

Agora, vamos configurar a página para ler essa busca e repassar para a sua API de restaurante.

1. Abra `src/app/cardapio/page.js`:

```
import Image from 'next/image';
import BotaoFavorito from "@components/BotaoFavorito";
import Link from 'next/link';
```

```

// [+] ADICIONADO: Importação do componente de busca
import Busca from "@components/Busca";

// [+] ADICIONADO: Recebendo searchParams para capturar a busca da URL
export default async function PaginaCardapio({ searchParams }) {

  // [+] ADICIONADO: Capturando o termo de busca (aguardando a Promise em versões
  novas do Next)
  const query = (await searchParams)?.busca || "";

  // 1. Buscando dados reais da sua API
  // Mantive o delay de 3s e o revalidate da versão original
  await new Promise((resolve) => setTimeout(resolve, 3000));

  const resposta = await fetch('https://api-restaurante-5iqb.onrender.com/api/produtos', {
    next: { revalidate: 60 }
  });

  const produtos = await resposta.json();

  // [+] ADICIONADO: Lógica de filtragem baseada no termo de busca
  const produtosFiltrados = produtos.filter(prato =>
    prato.nome.toLowerCase().includes(query.toLowerCase()) ||
    prato.descricao.toLowerCase().includes(query.toLowerCase())
  );

  return (
    <main className="p-10 max-w-7xl mx-auto">
      {/* Mantive o cabeçalho original com o contador de itens */}
      <div className="flex justify-between items-end mb-10">
        <div>
          <h1 className="text-4xl font-bold text-gray-800 font-serif">Nosso Cardápio</h1>
          <p className="text-gray-500 mt-2">Pratos artesanais preparados com ingredientes
frescos.</p>
        </div>
        <span className="text-sm bg-gray-100 px-3 py-1 rounded-full text-gray-400">
          {/* [-] REMOVIDO: produtos.length (original) */}
          {/* [+] ADICIONADO: produtosFiltrados.length para refletir a busca */}
          {produtosFiltrados.length} itens encontrados
        </span>
      </div>

      {/* [+] ADICIONADO: Componente de Busca entre o título e o grid */}
      <div className="mb-10">
        <Busca />
      </div>

      {/* [+] ADICIONADO: Verificação de lista vazia (Empty State) */}

```

```

{produtosFiltrados.length === 0 ? (
  <div className="text-center py-20 border-2 border-dashed border-gray-100
rounded-3xl">
    <p className="text-gray-400 text-xl">Nenhum prato encontrado para "{query}"</p>
    <Link href="/cardapio" className="text-orange-600 underline mt-2 block">Limpar
busca</Link>
  </div>
) : (
  <div className="grid grid-cols-1 md:grid-cols-2 lg:grid-cols-3 gap-8">
    { /* [-] REMOVIDO: produtos.map (original) */ }
    { /* [+] ADICIONADO: produtosFiltrados.map para exibir apenas o que foi buscado */ }
    { produtosFiltrados.map((prato) => (
      <div key={prato.id} className="bg-white rounded-2xl shadow-md overflow-hidden
flex flex-col border border-gray-100 hover:shadow-xl transition-shadow">

        <div className="relative h-56 w-full">
          <Image
            src={prato.imagem}
            alt={prato.nome}
            fill
            className="object-cover"
          />
          { /* Mantido funcionalidade de destaque original */ }
          { prato.destacado && (
            <span className="absolute top-4 left-4 bg-orange-600 text-white text-xs
font-bold px-3 py-1 rounded-full shadow-lg">
              ESTRELA DA CASA ★
            </span>
          ) }
        </div>

        <div className="p-6 flex flex-col flex-grow">
          <div className="flex justify-between items-start mb-2">
            { /* Mantido link para detalhes original */ }
            <Link href={` /cardapio/${prato.id}`} className="hover:opacity-95
transition-opacity">
              <h2 className="text-xl font-bold text-gray-800">{prato.nome}</h2>
            </Link>
            <span className="text-xs font-semibold text-orange-600 uppercase
tracking-wider">
              {prato.categoria}
            </span>
          </div>

          <p className="text-gray-500 text-sm mb-6 line-clamp-2">
            {prato.descricao}
          </p>

```

```

        <div className="mt-auto flex items-center justify-between border-t pt-4">
          <span className="text-2xl font-bold text-green-700">
            R$ {prato.preco.toFixed(2)}
          </span>
          <BotaoFavorito />
        </div>
      </div>
    </div>
  )))
</div>
)}
</main>
);
}

```

Caso apareça algum erro no <Image>, inserir o seguinte código no arquivo:
next.config.mjs

```

/** @type {import('next').NextConfig} */
const nextConfig = {
  images: {
    remotePatterns: [
      {
        protocol: 'https',
        hostname: 'unsplash.com',
      },
      {
        protocol: 'https',
        hostname: 'images.unsplash.com',
      },
    ],
  },
};

export default nextConfig;

```

Passo 3: Testando os Filtros

Realize os seguintes testes:

1. Digite "suco" no campo de busca. Observe a URL mudar para **?busca=suco** e a lista filtrar em tempo real.

2. Apague o texto. A lista deve voltar ao normal.
3. Copie a URL `localhost:3001/cardapio?busca=mignon` e cole em uma nova aba. O site já deve abrir filtrado.

Resumo

-  **searchParams**: A prop que as páginas de servidor usam para ler a URL.
-  **useSearchParams**: O hook que componentes de cliente usam para interagir com a URL.
-  **URLSearchParams**: Uma ferramenta nativa do JS para manipular parâmetros sem quebrar a URL.
-  **UX**: Filtros via URL são mais robustos e profissionais do que filtros apenas em memória (`useState`).

4.14 - Server Actions (POST)

Objetivo de Aprendizagem

Aprender a enviar dados do navegador para o servidor utilizando **Server Actions**, realizando o cadastro de novos pratos na API do restaurante de forma segura e eficiente.

Parte 1: Conceitual e Fundamentos

O que são Server Actions?

Imagine que o cliente do restaurante quer preencher uma ficha de sugestão. Em vez de ele levar a ficha até a recepção (API Route) para que alguém a leve para a cozinha, ele entrega a ficha diretamente na mão do chef (Server Action), que já está na cozinha pronto para processar.

Principais Vantagens:

- **Menos Código**: Você não precisa de `axios` ou `fetch` no lado do cliente para enviar o formulário. O Next.js gerencia a comunicação para você.
- **Segurança Nativa**: Como a função roda no servidor, você pode validar dados e usar chaves de API sem expô-las no navegador.

- **Revalidação Automática:** Com um único comando (`revalidatePath`), você avisa ao Next.js que a lista de pratos mudou e ele atualiza o cache do cardápio instantaneamente para todos.
-

Parte 2: Prática - Cadastrando um Novo Prato

Vamos criar uma área administrativa simples para adicionar pratos à sua API.

Passo 1: Criando a Action (A Lógica de Servidor)

Diferente das páginas, as ações costumam ficar em arquivos separados para serem reutilizadas.

1. Crie o arquivo `src/app/admin/actions.js`:

"use server"; //  OBRIGATÓRIO: Indica que todas as funções aqui rodam no servidor

```
import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";
```

```
export async function cadastrarPrato(formData) {
  // 1. Extraímos os dados do formulário
  const dados = {
    nome: formData.get("nome"),
    descricao: formData.get("descricao"),
    categoria: formData.get("categoria"),
    preco: parseFloat(formData.get("preco")),
    imagem: formData.get("imagem"),
    destacado: formData.get("destacado") === "on",
  };

  // 2. Enviamos para a sua API Real (POST)
  const res = await fetch("https://api-restaurant-5iqb.onrender.com/api/produtos", {
    method: "POST",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(dados),
  });

  if (!res.ok) {
    throw new Error("Falha ao cadastrar o prato na API.");
  }

  // 3. Limpamos o cache da página de cardápio para o novo prato aparecer
  revalidatePath("/cardapio");
}
```

```
// 4. Redirecionamos o usuário de volta
redirect("/cardapio");
}
```

Passo 2: Criando o Formulário de Cadastro

Vamos criar a página onde o gerente do restaurante preencherá os dados.

1. Crie a pasta `src/app/admin/novo-prato`.
2. Crie o arquivo `page.js` dentro dela:

```
import { cadastrarPrato } from "../actions";

export default function PaginaNovoPrato() {
  return (
    <main className="max-w-2xl mx-auto p-10 bg-white shadow-xl rounded-3xl mt-10">
      <h1 className="text-3xl font-bold mb-6 text-gray-800">Novo Prato no Cardápio</h1>

      {/* 💡 A "mágica": passamos a Action diretamente para o atributo 'action' */}
      <form action={cadastrarPrato} className="flex flex-col gap-4">
        <input name="nome" placeholder="Nome do Prato" required className="p-3 border
rounded-xl" />

        <textarea name="descricao" placeholder="Descrição completa" required
className="p-3 border rounded-xl h-32" />

        <select name="categoria" required className="p-3 border rounded-xl bg-white">
          <option value="Entradas">Entradas</option>
          <option value="Pratos Principais">Pratos Principais</option>
          <option value="Sobremesas">Sobremesas</option>
          <option value="Bebidas">Bebidas</option>
        </select>

        <input name="preco" type="number" step="0.01" placeholder="Preço (R$)" required
className="p-3 border rounded-xl" />

        <input name="imagem" type="url" placeholder="URL da imagem (Unsplash)" required
className="p-3 border rounded-xl" />

        <label className="flex items-center gap-2 cursor-pointer p-2">
          <input name="destacado" type="checkbox" className="w-5 h-5 accent-orange-600"
/>
          <span className="text-gray-700">Destaque da Casa?</span>
        </label>
```

```

        <button type="submit" className="bg-orange-600 text-white p-4 rounded-xl font-bold
hover:bg-orange-700 transition-all">
            Salvar no Cardápio
        </button>
    </form>
</main>
);
}

```

Passo 3: O Teste de Fluxo

vamos testar o fluxo:

1. Acessar `/admin/novo-prato`.
 2. Preencher com um nome (ex: "Pizza de Pepperoni"), uma URL de imagem do Unsplash e o preço.
 3. Clicar em "Salvar".
 4. **O que vai acontecer?** O navegador enviará os dados, a Action fará o POST para a sua API, o cache será limpo e o aluno será jogado de volta para `/cardapio`, onde o novo prato já deve aparecer na lista.
-

Comparativo: POST Tradicional vs Server Action

Recurso	POST com API Route (Antigo)	Server Actions (2026)
Endpoint	Precisa criar <code>/api/produtos/route.js</code>	Não precisa de endpoint manual
Envio	<code>onSubmit</code> + <code>fetch('/api/...')</code>	Atributo <code>action</code> nativo do HTML
Estado	Precisa de <code>useState</code> para cada campo	Usa o objeto nativo <code>formData</code>
Cache	Precisa de <code>router.refresh()</code> ou recarregar	<code>revalidatePath</code> faz tudo automático

Resumo

-  **"use server"**: Define que a lógica nunca será vista pelo cliente.
-  **formData**: Forma nativa do navegador de agrupar dados de formulário.
-  **revalidatePath**: A ferramenta mais importante para manter o site atualizado sem refresh manual.

-  **Progressive Enhancement:** Server Actions funcionam até se o JavaScript do usuário estiver desabilitado (em formulários simples).

4.15 - Mutação de Dados: Editar e Excluir

Objetivo de Aprendizagem

Aprender a manipular dados existentes na API utilizando os métodos **PUT** e **DELETE**, integrando-os com Server Actions e garantindo a atualização instantânea da interface.

Parte 1: Conceitual e Fundamentos

O Ciclo de Vida da Edição

Editar um prato é um processo de dois passos:

1. **Busca (GET):** Primeiro, precisamos carregar os dados atuais do prato para preencher o formulário. O usuário não deve digitar tudo de novo.
2. **Atualização (PUT):** Após a alteração, enviamos apenas o que mudou (ou o objeto inteiro atualizado) para o servidor.

A Exclusão Segura

Excluir dados é uma ação destrutiva. No Next.js, usamos Server Actions para garantir que a exclusão ocorra no servidor, mas geralmente adicionamos uma pequena camada de interatividade no cliente (um "Tem certeza?") para evitar cliques acidentais.

Parte 2: Prática - Implementando no Restaurante

Vamos atualizar nosso arquivo de ações e criar as novas interfaces.

Passo 1: Atualizando as Ações (`src/app/admin/actions.js`)

Vamos adicionar as funções `editarPrato` e `excluirPrato` ao arquivo que criamos no tópico anterior.

```

"use server";

import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";

// --- AÇÃO DE EDITAR ---
export async function editarPrato(id, formData) {
  const dados = {
    nome: formData.get("nome"),
    descricao: formData.get("descricao"),
    categoria: formData.get("categoria"),
    preco: parseFloat(formData.get("preco")),
    imagem: formData.get("imagem"),
    destacado: formData.get("destacado") === "on",
  };

  await fetch(`https://api-restaurante-5iqb.onrender.com/api/produtos/${id}`, {
    method: "PUT",
    headers: { "Content-Type": "application/json" },
    body: JSON.stringify(dados),
  });

  revalidatePath("/cardapio");
  revalidatePath(`/cardapio/${id}`);
  redirect("/cardapio");
}

// --- AÇÃO DE EXCLUIR ---
export async function excluirPrato(id) {
  await fetch(`https://api-restaurante-5iqb.onrender.com/api/produtos/${id}`, {
    method: "DELETE",
  });

  revalidatePath("/cardapio");
  // Não usamos redirect aqui pois a ação será chamada de dentro da lista
}

```

Passo 2: O Botão de Excluir (Client Component)

Para ter um alerta de confirmação, este botão precisa ser um Client Component.

1. Crie `src/components/BotaoExcluir.js`:

JavaScript

```
"use client";
```

```
import { excluirPrato } from "@app/admin/actions";

export default function BotaoExcluir({ id }) {
  return (
    <button
      onClick={async () => {
        if (confirm("Deseja realmente remover este prato do cardápio?")) {
          await excluirPrato(id);
        }
      }}
      className="text-red-500 hover:text-red-700 text-sm font-bold"
    >
      Excluir
    </button>
  );
}
```

Passo 3: A Página de Edição (Dynamic Route)

Vamos criar uma rota para editar o prato: `/admin/editar/[id]`.

1. Crie a pasta `src/app/admin/editar/[id]/page.js`:

```
import { editarPrato } from "../../actions";

export default async function PaginaEditarPrato({ params }) {
  const { id } = await params;

  // 1. Busca os dados atuais do prato para preencher o form
  const res = await fetch(`https://api-restaurante-5iqb.onrender.com/api/produtos/${id}`);
  const prato = await res.json();

  // 2. Criamos uma versão da Action que já conhece o ID (usando bind)
  const editarPratoComId = editarPrato.bind(null, id);

  return (
    <main className="max-w-2xl mx-auto p-10 bg-white shadow-xl rounded-3xl mt-10">
      <h1 className="text-3xl font-bold mb-6 text-gray-800">Editar Prato</h1>

      <form action={editarPratoComId} className="flex flex-col gap-4">
        <input name="nome" defaultValue={prato.nome} className="p-3 border rounded-xl" />
        <textarea name="descricao" defaultValue={prato.descricao} className="p-3 border
rounded-xl h-32" />
      </form>
    </main>
  );
}
```

```

    <select name="categoria" defaultValue={prato.categoria} className="p-3 border
rounded-xl bg-white">
      <option value="Entradas">Entradas</option>
      <option value="Pratos Principais">Pratos Principais</option>
      <option value="Sobremesas">Sobremesas</option>
      <option value="Bebidas">Bebidas</option>
    </select>

    <input name="preco" type="number" step="0.01" defaultValue={prato.preco}
className="p-3 border rounded-xl" />
    <input name="imagem" type="url" defaultValue={prato.imagem} className="p-3
border rounded-xl" />

    <label className="flex items-center gap-2 p-2">
      <input name="destacado" type="checkbox" defaultChecked={prato.destacado}
className="w-5 h-5 accent-orange-600" />
      <span className="text-gray-700">Manter em Destaque?</span>
    </label>

    <button type="submit" className="bg-blue-600 text-white p-4 rounded-xl font-bold
hover:bg-blue-700">
      Atualizar Prato
    </button>
  </form>
</main>
);
}

```

Passo 4: O Painel Administrativo (<src/app/admin/page.js>)

Esta página herda a lógica de busca do cardápio, mas renderiza uma interface de controle.

```

import Image from 'next/image';
import Link from 'next/link';
import Busca from "@/components/Busca";
import BotaoExcluir from "@/components/BotaoExcluir"; // Componente Client

export default async function PaginaAdmin({ searchParams }) {
  // [+] ADICIONADO: Captura de busca idêntica ao cardápio
  const query = (await searchParams)?.busca || "";

  const resposta = await fetch('https://api-restaurant-5iqb.onrender.com/api/produtos', {
    next: { revalidate: 0 } // Admin precisa de dados sempre frescos
  });
  const produtos = await resposta.json();

```

```

// [+] ADICIONADO: Filtro para facilitar encontrar o prato a editar
const produtosFiltrados = produtos.filter(prato =>
  prato.nome.toLowerCase().includes(query.toLowerCase())
);

return (
  <main className="p-10 max-w-6xl mx-auto">
    <div className="flex justify-between items-center mb-8">
      <div>
        <h1 className="text-3xl font-bold text-gray-800">Gerenciar Cardápio</h1>
        <p className="text-gray-500">Edite ou remova pratos da base de dados.</p>
      </div>
      { /* Link para criar novo prato (visto no tópico anterior) */ }
      <Link href="/admin/novo-prato" className="bg-green-600 text-white px-6 py-2
rounded-lg font-bold hover:bg-green-700 transition-colors">
        + Novo Prato
      </Link>
    </div>

    <div className="mb-6">
      <Busca />
    </div>

    <div className="bg-white shadow-md rounded-xl overflow-hidden border
border-gray-200">
      <table className="w-full text-left">
        <thead className="bg-gray-50 border-b border-gray-200">
          <tr>
            <th className="p-4 font-semibold text-gray-600">Prato</th>
            <th className="p-4 font-semibold text-gray-600">Categoria</th>
            <th className="p-4 font-semibold text-gray-600 text-right">Ações</th>
          </tr>
        </thead>
        <tbody>
          {produtosFiltrados.map((prato) => (
            <tr key={prato.id} className="border-b last:border-none hover:bg-gray-50
transition-colors">
              <td className="p-4 flex items-center gap-4">
                <div className="relative h-12 w-12 rounded-lg overflow-hidden border">
                  <Image src={prato.imagem} alt={prato.nome} fill className="object-cover" />
                </div>
                <span className="font-medium text-gray-800">{prato.nome}</span>
              </td>
              <td className="p-4">
                <span className="text-xs bg-orange-100 text-orange-700 px-2 py-1 rounded-full
font-bold uppercase">
                  {prato.categoria}
                </span>
              </td>
            </tr>
          ))}
        </tbody>
      </table>
    </div>
  </main>
);

```

```

</td>
<td className="p-4 text-right">
  <div className="flex justify-end gap-4">
    { /* [+] ADICIONADO: Botão de Edição (Link) */ }
    <Link
      href={` /admin/editar/${prato.id}`}
      className="text-blue-600 hover:text-blue-800 font-semibold text-sm"
    >
      Editar
    </Link>

    { /* [+] ADICIONADO: Componente de Exclusão com confirmação */ }
    <BotaoExcluir id={prato.id} />
  </div>
</td>
</tr>
  )}
</tbody>
</table>

{produtosFiltrados.length === 0 && (
  <div className="p-10 text-center text-gray-400">
    Nenhum prato encontrado para "{query}".
  </div>
)}
</div>
</main>
);
}

```

Passo 5: Teste de Fluxo Completo

Vamos realizar a "Prova Real":

1. **Excluir:** Clique em excluir em um prato. O alerta deve aparecer, e ao confirmar, o prato deve sumir da lista instantaneamente (graças ao `revalidatePath`).
2. **Editar:** Clique em editar. O formulário deve abrir já preenchido com os dados do prato. Mude o preço e salve. Você deve ser redirecionado e ver o novo preço na lista.

Resumo

-  **PUT e DELETE:** Usamos os métodos HTTP corretos para cada intenção.

-  **bind()**: Técnica para passar o ID do prato para a Server Action sem precisar de campos ocultos no formulário.
-  **defaultValue**: Em formulários de edição, usamos **defaultValue** em vez de **value** para permitir que o usuário digite sem precisar de um **useState** para cada campo.
-  **Cache inteligente**: O **revalidatePath** garante que a lista e a página de detalhes reflitam a mudança imediatamente.

4.16 - Middleware: Proteção de Rotas e Segurança

Objetivo de Aprendizagem

Aprender a utilizar o **Middleware** do Next.js para interceptar requisições e proteger áreas sensíveis (como o Painel Admin), garantindo que apenas usuários autorizados possam gerenciar o cardápio.

Parte 1: Conceitual e Fundamentos

O que é o Middleware?

Imagine que o seu restaurante tem uma área VIP ou um escritório nos fundos. O **Middleware** é a segurança que fica no corredor. Antes de você chegar à porta da sala, ele te para e pergunta: *"Você tem autorização?"*.

- Se **sim**, ele te deixa passar.
- Se **não**, ele te redireciona imediatamente para a recepção (Página de Login).

Por que usar Middleware em vez de checar em cada página?

1. **Centralização**: Você escreve a regra de segurança em um único arquivo, em vez de repetir o código em todas as páginas do admin.
 2. **Velocidade**: Ele roda **antes** de qualquer página ser renderizada. O usuário nem chega a baixar o conteúdo da página se não tiver permissão.
 3. **Filtro Global**: Você pode decidir exatamente quais rotas proteger usando um "Matcher" (filtro).
-



Parte 2: Prática - Bloqueando o Acesso ao Admin

Vamos simular um sistema de autenticação simples baseado em **Cookies**. O objetivo é impedir que o usuário acesse qualquer rota que comece com `/admin` sem estar "logado".



Passo 1: Criando o arquivo `middleware.js`

Este arquivo é especial e deve ser criado obrigatoriamente na raiz do seu projeto (dentro de `src/` se você estiver usando essa pasta).

1. Crie o arquivo `src/middleware.js`:

```
import { NextResponse } from 'next/server';

export function middleware(request) {
  // 1. Buscamos um cookie chamado 'auth_token' (simulando login)
  const token = request.cookies.get('auth_token')?.value;

  // 2. Se o usuário tentar acessar o /admin e NÃO tiver o token
  if (request.nextUrl.pathname.startsWith('/admin') && !token) {
    // Redirecionamos para a página de login
    return NextResponse.redirect(new URL('/login', request.url));
  }

  // 3. Se estiver tudo ok, ele segue viagem
  return NextResponse.next();
}

// 4. Configuramos quais rotas o middleware deve vigiar
export const config = {
  matcher: '/admin/:path*', // Protege /admin e qualquer sub-rota (editar, novo, etc.)
};
```



Passo 2: Criando a Página de Login (Simulação)

Para podermos testar a entrada no sistema, precisamos de uma página que "crie" esse cookie de acesso.

1. Crie a pasta `src/app/login/`.
2. Crie o arquivo `page.js`:

```
"use client";

export default function PaginaLogin() {
  function fazerLogin() {
```



```
// Simulando a criação de um cookie de acesso que expira em 1 hora
document.cookie = "auth_token=logado; path=/; max-age=3600";
alert("Acesso autorizado! Redirecionando...");
window.location.href = "/admin";
}

return (
  <main className="flex flex-col items-center justify-center min-h-[70vh]">
    <div className="bg-white p-10 rounded-3xl shadow-xl border border-gray-100
text-center">
      <h1 className="text-3xl font-bold mb-4">Acesso Restrito</h1>
      <p className="text-gray-500 mb-8">Área exclusiva para gerentes do restaurante.</p>

      <button
        onClick={fazerLogin}
        className="bg-orange-600 text-white px-10 py-4 rounded-xl font-bold
hover:bg-orange-700 transition-all"
        >
        Simular Login de Gerente
      </button>
    </div>
  </main>
);
}
```

Passo 3: Teste de Segurança

Faça o seguinte teste:

1. Tente acessar diretamente `localhost:3001/admin`.
 2. **O que deve acontecer?** O Middleware vai te "expulsar" e te jogar para a página `/login`.
 3. Na página de login, clique no botão para simular o acesso.
 4. Tente acessar o `/admin` novamente.
 5. **O que deve acontecer?** Agora o segurança te deixa passar e você consegue ver a tabela de pratos!
-

Resumo

-  **Middleware:** Roda no "limiar" entre o pedido do usuário e a resposta do servidor.
-  **Config Matcher:** Permite definir exatamente quais URLs precisam de proteção (ex: `/admin/:path*`).
-  **Cookies:** São a forma mais comum de persistir o estado de login no Next.js devido à facilidade de leitura no servidor.
-  **NextResponse:** Objeto usado para redirecionar o usuário ou permitir que ele continue.

4.17 - Metadata e SEO: O Restaurante no Topo

Objetivo de Aprendizagem

Dominar a **Metadata API** do Next.js para configurar títulos, descrições e imagens de compartilhamento (Open Graph) de forma estática e dinâmica, garantindo que cada prato do cardápio seja um "imã" de cliques.

Parte 1: Conceitual e Fundamentos

O que é Metadata?

Metadata são informações "invisíveis" que ficam dentro da tag `<head>` do seu HTML. Elas dizem aos robôs (Google, Bing, Facebook) do que se trata a sua página.

- **Title:** O nome que aparece na aba do navegador e no título da busca.
- **Description:** Aquele pequeno resumo cinza que aparece no Google.
- **Open Graph (OG):** Define qual imagem, título e descrição aparecem quando você cola o link no WhatsApp.

Estático vs. Dinâmico

1. **Estático:** Para páginas que não mudam (Home, Sobre). Você escreve o texto uma vez e pronto.
 2. **Dinâmico:** Para páginas de pratos. Se o usuário abrir a página do "Petit Gâteau", o título da aba deve ser "Petit Gâteau - CentroWEG Food", não apenas "Restaurante".
-

Parte 2: Prática - Aplicando no Projeto

O Next.js facilita isso permitindo que exportemos um objeto chamado `metadata` ou uma função chamada `generateMetadata`.

🔧 Passo 1: Metadata Global (`src/app/layout.js`)

Aqui definimos o "padrão" do site. Se uma página não tiver um título próprio, ela usará este.

```
// src/app/layout.js
```

```
export const metadata = {
  title: "Nosso Restaurante",
  description: "Cardápio digital do restaurante",
};

export default function RootLayout({ children }) {

  return (
    <html lang="pt-br">
      <body className={inter.className}>{children}</body>
    </html>
  );
}
```

Ficando assim:

```
import "../globals.css";
// Importamos a fonte do Google Fonts através da biblioteca de
// otimização de fontes do Next.js
import { Inter } from "next/font/google";

// Instanciamos e configuramos a fonte Inter
const inter = Inter({
  // 'subsets': Define quais conjuntos de caracteres (acentos/letras)
  // serão baixados.
  // O valor ["latin"] reduz o tamanho da fonte ao baixar apenas os
  // caracteres do nosso idioma.
  subsets: ["latin"],

  // 'display': Define o comportamento de renderização do texto enquanto
  // a fonte carrega.
  // 'swap' instrui o navegador a exibir imediatamente uma fonte do
  // sistema até que a fonte personalizada esteja pronta, evitando texto
  // invisível.
  display: "swap"
```

```

});

export const metadata = {
  title: "Nosso Restaurante",
  description: "Cardápio digital do restaurante",
};

// 'RootLayout': Componente de layout raiz da aplicação.
// Ele envolve todas as páginas do sistema, definindo a estrutura HTML
base (<html> e <body>) e elementos globais como fontes e navegação.
export default function RootLayout({ children }) {
  // 'children': Propriedade especial do React que representa o conteúdo
  visual da página/rota atual sendo acessada (ex: page.js de /cardapio ou
  /admin).
  return (
    <html lang="pt-br">
      {/* Aplicamos a classe gerada pela fonte na tag <body> para
      alterar toda a aplicação */}
      <body className={inter.className}>{children}</body>
    </html>
  );
}

```

Passo 2: Metadata Estática (src/app/cardapio/page.js)

Para a listagem geral, queremos algo chamativo.

```
// src/app/cardapio/page.js
```

```

export const metadata = {
  title: "Cardápio Completo",
  description: "Explore nossa seleção de Entradas, Pratos Principais e Sobremesas
  exclusivas.",
};

export default async function PaginaCardapio() {
  /* ... resto do código ... */
}

```

Passo 3: Metadata Dinâmica (src/app/cardapio/[id]/page.js)

Este é o "pulo do gato". Vamos buscar o nome do prato na API para colocar no título da página.

```
// src/app/cardapio/[id]/page.js
```

```
// 💡 A função generateMetadata roda no servidor antes da página carregar
export async function generateMetadata({ params }) {
  const { id } = await params;

  // Buscamos os dados do prato para usar no SEO
  const res = await fetch(`https://api-restaurante-5iqb.onrender.com/api/produtos/${id}`);
  const prato = await res.json();

  return {
    title: prato.nome,
    description: prato.descricao,
    openGraph: {
      images: [prato.imagem], // A foto do prato aparece no card do WhatsApp!
    },
  };
}

export default async function DetalhePrato({ params }) {
  /* ... resto do código ... */
}
```

Passo 4: O Teste de SEO

Vamos testar:

1. Abrir a página de um prato específico no navegador.
2. Olhar o título na aba do navegador. **O que deve acontecer?** Deve aparecer algo como: **Bruschetta Caprese | Sabor e Arte**.
3. Inspeccionar o código-fonte (Ctrl+U) e procurar por `<title>` e `<meta name="description">`.

Resumo para o Notion

-  **Metadata API:** Substitui a manipulação manual da tag `<head>`.
-  **template:** No `layout.js`, ajuda a manter a marca do restaurante em todas as abas.

-  **generateMetadata**: Essencial para rotas dinâmicas; permite que cada prato tenha seu próprio "cartão de visitas" na web.
-  **Social Media**: Usar a imagem da API no **openGraph** aumenta drasticamente as chances de alguém clicar no link compartilhado.

4.18 - Estilização: Tailwind CSS 4 e CSS Modules

Objetivo

Dominar as duas principais formas de estilização no Next.js: a abordagem utilitária com **Tailwind CSS 4** e a abordagem encapsulada com **CSS Modules**, escolhendo a melhor ferramenta para cada parte do projeto **Sabor & Arte**.

Parte 1: Conceitual e Fundamentos

A estilização no Next.js evoluiu para priorizar a performance e a manutenção. No projeto, utilizaremos o Tailwind para a estrutura ágil e o CSS Puro (Modules) para componentes que exigem um design altamente específico ou animações complexas.

1. Tailwind CSS 4 (Utility-First)

O Tailwind 4 simplifica drasticamente a configuração. Em vez de arquivos de configuração gigantescos, ele utiliza diretivas de CSS moderno.

- **Vantagem**: Desenvolvimento ultra rápido, sem sair do arquivo HTML/JSX.
- **Performance**: Gera um arquivo CSS final contendo apenas as classes realmente utilizadas.

2. CSS Modules (Scoped CSS)

O Next.js suporta CSS Modules nativamente usando a extensão **.module.css**.

- **Vantagem**: O CSS é exclusivo daquele componente. Não há conflito de nomes (o Next.js gera classes únicas como **Botao_sucesso__z3x4**).
- **Uso Ideal**: Estilização de elementos complexos onde o Tailwind ficaria muito poluído.

Recurso	Tailwind CSS 4	CSS Modules
Curva de Aprendizado	Baixa (após decorar as classes)	Baixa (CSS Padrão)
Manutenibilidade	Alta (classes no mesmo arquivo)	Média (arquivos separados)
Performance	Excelente (Zero runtime JS)	Excelente (CSS estático)
Encapsulamento	Por convenção	Nativo e automático

Parte 2: Prática - Estilizando o Sabor & Arte

Passo 1: Configuração do Tailwind 4

No Tailwind 4, a configuração é feita diretamente no seu arquivo de entrada de CSS.

1. Abra o arquivo `src/app/globals.css`.
2. Substitua todo o conteúdo pela nova diretiva de importação única:

```
@import "tailwindcss";
```

```
/* Aqui você pode adicionar camadas personalizadas se necessário */
```

```
@theme {  
  --color-brand-orange: #f97316;  
  --color-brand-green: #15803d;  
}
```

Passo 2: Estilização Utilitária (Tailwind)

Vamos estilizar o componente de exibição de pratos do **Sabor & Arte** focando em um design limpo e moderno.

1. Abra (ou crie) o componente `src/components/PratoCard.js`:

```
import Image from 'next/image';
```

```
export default function PratoCard({ prato }) {  
  return (  
    <div className="group overflow-hidden rounded-3xl border border-gray-100 bg-white  
shadow-sm transition-all hover:shadow-xl">  
      <div className="relative h-64 w-full">  
        <Image  
          src={prato.imagem}  
          alt={prato.nome}
```

```

    fill
    className="object-cover transition-transform duration-500 group-hover:scale-110"
  />
  <div className="absolute top-4 left-4 rounded-full bg-white/90 px-3 py-1 text-xs
font-bold text-brand-orange shadow-sm backdrop-blur-md">
    {prato.categoria}
  </div>
</div>

<div className="p-6">
  <h3 className="text-xl font-bold text-gray-900">{prato.nome}</h3>
  <p className="mt-2 text-sm text-gray-500 line-clamp-2">{prato.descricao}</p>

  <div className="mt-6 flex items-center justify-between">
    <span className="text-2xl font-black text-brand-green">
      R$ {prato.preco.toFixed(2)}
    </span>
    <button className="rounded-xl bg-gray-900 px-5 py-2.5 text-sm font-bold text-white
transition-colors hover:bg-brand-orange">
      Ver Detalhes
    </button>
  </div>
</div>
</div>
);
}

```

Passo 3: Estilização Encapsulada (CSS Modules)

Para elementos que exigem efeitos personalizados, como um gradiente animado no topo do site, usaremos CSS puro.

1. Crie o arquivo `src/components/Banner.module.css`:

```

.hero {
  background: linear-gradient(45deg, #f97316, #fbbf24);
  padding: 4rem 2rem;
  border-radius: 2rem;
  text-align: center;
  position: relative;
  overflow: hidden;
}

.title {
  color: white;
  font-size: 3rem;
}

```



```
font-weight: 900;
text-shadow: 0 4px 6px rgba(0, 0, 0, 0.1);
}
```

1. Aplique no componente `src/components/Banner.js`:

```
import styles from './Banner.module.css';
```

```
export default function Banner() {
  return (
    <section className={styles.hero}>
      <h1 className={styles.title}>Sabor & Arte</h1>
      <p className="text-white/90 font-medium">Experiências gastronômicas
inesquecíveis.</p>
    </section>
  );
}
```

✓ Resumo da Estratégia

- **Global:** Use o `globals.css` apenas para o reset do Tailwind 4 e variáveis de tema.
- **Componentes de Interface:** Use Tailwind para 90% do projeto. Ele mantém o código conciso e facilita a responsividade.
- **Componentes Complexos:** Use CSS Modules quando as classes do Tailwind começarem a dificultar a leitura do código ou quando precisar de keyframes de animação CSS puras.

4.19 - Arquitetura de Pastas e Organização Profissional

A organização de pastas é o "esqueleto" que sustenta a escalabilidade de um software. No Next.js, com o **App Router**, a estrutura ganhou importância dobrada: as pastas não apenas organizam arquivos, mas definem as **rotas** e a **estratégia de renderização** da aplicação.

Para o projeto **Sabor & Arte**, exploramos as três abordagens mais respeitadas pelo mercado, indo da simplicidade à robustez extrema.

1. Estrutura por Camadas (Layered Architecture)

Esta é a abordagem clássica e a porta de entrada para quem busca o **Clean Code** (Código Limpo). A ideia principal é o **SoC (Separation of Concerns)**: separar o código por "o que ele é".

A Estrutura Sugerida:

```
src/
├── app/           # Rotas, Layouts e Server Components
├── components/    # Componentes visuais reutilizáveis (Botao, Card, Navbar)
├── hooks/         # Lógicas de estado personalizadas (useCarrinho, useAuth)
├── services/      # Chamadas de API e integrações (api.js)
├── utils/         # Funções auxiliares (formatarPreco, validarData)
├── constants/     # Valores fixos e Mocks (pratos.js)
└── types/        # Definições de tipos (caso use TypeScript)
```

Por que usar? (Clean Code & SoC)

É uma estrutura intuitiva. Se é necessário encontrar a função que formata a moeda no cardápio, o destino é a pasta **utils**. Se o objetivo é alterar o estilo do botão de "Pedir", ele estará em **components**. É ideal para projetos de pequeno e médio porte onde a agilidade é prioridade.

2. Estrutura por Módulos (Feature-Based)

À medida que o sistema cresce — adicionando área administrativa, sistema de pagamentos e rastreamento de pedidos — a estrutura por camadas pode se tornar um labirinto. Aqui entra o conceito de **Domain-Driven Design (DDD)** adaptado para o Front-end.

A Estrutura Sugerida:

```
src/
├── app/           # Apenas a definição das rotas e pontos de entrada
├── modules/       # O "coração" do negócio organizado por domínio
│   ├── cardapio/  # Componentes, hooks e lógica exclusivos do cardápio
│   ├── pedidos/   # Tudo sobre o carrinho e finalização de compras
│   └── auth/      # Login, registro e proteção de acesso
├── shared/        # Recursos globais usados por todos os módulos
│   ├── components/ # UI Genérica (Inputs, Modais)
│   └── utils/
└──
```

Por que usar? (DDD & Encapsulamento)

O código é organizado em torno das funcionalidades do negócio. Se o módulo de "pedidos" apresentar uma falha, o código do "cardápio" permanece isolado e intacto. Isso facilita a manutenção e evita o surgimento de efeitos colaterais em partes não relacionadas do sistema.

3. Design Atômico (Atomic Design)

Esta metodologia, criada por Brad Frost, foca exclusivamente na **Interface do Usuário (UI)**. É a escolha preferida de empresas que trabalham com sistemas de design (Design Systems) rigorosos.

A Estrutura Sugerida (em **src/components**):

```
components/
├── atoms/      # Elementos básicos e indivisíveis (Label, Input, Botao)
├── molecules/  # Grupos de átomos (Campo de Busca, Item do Carrinho)
├── organisms/  # Seções complexas (Navbar, Card do Prato, Footer)
└── templates/  # Estruturas de layout da página
```

Por que usar? (Reutilização Máxima)

Promove uma consistência visual absoluta. Um "Átomo" de botão é usado em uma "Molécula" de busca, que por sua vez compõe o "Organismo" da Navbar. Qualquer alteração na base (Átomo) reflete instantaneamente em todo o ecossistema do **Sabor & Arte**.

Recomendação Prática para o Sabor & Arte

Para equilibrar velocidade de entrega e organização, a recomendação é utilizar uma estrutura híbrida de **Camadas** com o **App Router**.

Organização Final do Projeto:

```
src/
├── app/          # Roteamento (Pastas = URLs)
│   ├── globals.css  # Configuração Tailwind 4 (@import "tailwindcss";)
│   ├── layout.js    # Moldura global (Navbar/Footer)
│   └── page.js      # Página inicial
├── components/    # Componentes Visuais
│   ├── ui/        # Componentes de base (Botao, Input)
│   └── layout/     # Componentes de estrutura (Navbar, Sidebar)
├── lib/          # Configurações de serviços externos (API, Banco)
└── assets/       # Mídias locais (Logos, Ícones)
```

Nota Técnica: Lembre-se que no **Tailwind CSS 4**, não há necessidade de arquivos de configuração complexos. O arquivo `globals.css` torna-se o cérebro da estilização, onde as variáveis de tema são definidas e o framework é importado em uma única linha.

Regras de Ouro

1. **Mantenha o `app/` enxuto:** Use as pastas dentro de `app/` apenas para arquivos especiais (`page.js`, `layout.js`). A lógica e os componentes visuais devem morar em pastas dedicadas fora do roteamento.
2. **Consistência de Nomenclatura:** Escolha um padrão (ex: inglês para nomes de pastas técnicos como `hooks` e `services`) e mantenha-o em todo o projeto.
3. **Encapsulamento:** Se um componente é usado apenas em uma página específica, ele pode morar próximo a ela. Se for usado em mais de um lugar, mova-o para a pasta global de `components`.